

Pre-entrega 6: Analitica avanzada in-stream con Redshift

Streaming Ingestion + Materialized Views + Lakehouse Iceberg

Alumno: Gustavo Pereyra

Proyecto: CD_DE | AWS us-east-1 | Septiembre 2026

Estado del documento: Version final. La infraestructura fue destruida con Terraform y se verifico posteriormente que Redshift Serverless, Managed Flink y Kinesis ya no existian.

Objetivo

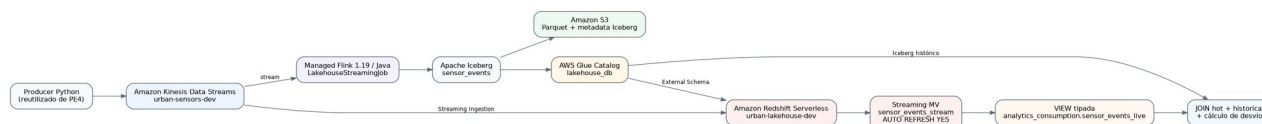
Integrar una capa analitica de baja latencia en Amazon Redshift Serverless con el pipeline Kinesis + Flink + Iceberg construido en entregas anteriores.

1. Resumen ejecutivo

La solución implementa un patrón híbrido de datos calientes e históricos. Kinesis Data Streams alimenta en paralelo el procesamiento persistente en Flink/Iceberg y una capa de Redshift Streaming Ingestion de baja latencia. Redshift consume el stream mediante una Materialized View con AUTO REFRESH YES, expone una vista tipada para analítica, consulta el histórico Iceberg mediante Glue Data Catalog y combina ambas capas en una misma sentencia SQL.

- Infraestructura desplegada con Terraform: 22 recursos planificados en total; 1 bucket creado de forma controlada y 21 recursos agregados en el apply completo.
- Redshift Serverless: namespace/workgroup urban-lakehouse-dev, base dev, capacidad base 8 RPU.
- Streaming Ingestion directa desde urban-sensors-dev, sin S3 intermedio para la ruta de baja latencia.
- JSON modelado como SUPER y luego convertido a columnas nativas: sensor_id, event_time, temperature, humidity y air_quality_index.
- Auto refresh validado por crecimiento del conteo de 899 a 1088 eventos sin INSERT manual en Redshift.
- Integración hot + historical validada mediante JOIN con lakehouse_iceberg.sensor_events.
- Seguridad: IAM de mínimo privilegio, VPC endpoint de Kinesis, require_ssl=true y RBAC SQL con analytics_reader.
- Monitoreo: SYS_STREAM_SCAN_STATES, SYS_STREAM_SCAN_ERRORS y SYS_MV_REFRESH_HISTORY; la consulta de errores devolvió 0 filas.
- Cierre: terraform destroy eliminó los 22 recursos; terraform state list quedó vacío y las comprobaciones AWS CLI devolvieron ResourceNotFoundException para Redshift, Flink y Kinesis.

2. Arquitectura actualizada



Arquitectura final de la Pre-entrega 6: doble camino hot/historical y convergencia analítica en Redshift.

El diseño preserva el Lakehouse de la Pre-entrega 5 como sistema histórico y agrega Redshift como motor de consulta operacional. Este patrón evita reemplazar Iceberg por Redshift: ambas capas cumplen funciones complementarias de persistencia/replay y baja latencia.

3. Infraestructura Terraform y Redshift Serverless

La Pre-entrega 6 reutiliza los modulos de Kinesis y Flink de entregas anteriores y agrega recursos especificos de Redshift Serverless. La region se mantiene en us-east-1 para conservar proximidad y compatibilidad con Kinesis, Glue y S3 existentes.

Componente	Implementacion
IAM Redshift	Rol redshift-streaming-ingestion-dev con lectura de Kinesis, Glue y S3 Lakehouse.
Red privada	Interface VPC Endpoint para Kinesis, private_dns_enabled=true.
Redshift Serverless	Namespace y workgroup urban-lakehouse-dev; db dev; base/max capacity 8.
Seguridad de transporte	enhanced_vpc_routing=true y require_ssl=true.
Catalogo historico	Glue database lakehouse_db y objetos Iceberg en S3.

```
Plan: 22 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ artifact_bucket_name      = "coderhouse-urban-streaming-raw-gusper-dev"
+ firehose_arn              = (known after apply)
+ firehose_name             = "ingesta-urban-sensors-dev"
+ flink_application_name    = "urban-stream-processing-dev"
+ glue_database_name       = "lakehouse_db"
+ lakehouse_bucket_arn     = (known after apply)
+ lakehouse_bucket_name    = "coderhouse-urban-lakehouse-gusper-dev"
+ redshift_iam_role_arn     = (known after apply)
+ redshift_namespace_name  = "urban-lakehouse-dev"
+ redshift_workgroup_endpoint = (known after apply)
+ redshift_workgroup_name   = "urban-lakehouse-dev"
+ stream_arn               = (known after apply)
+ stream_name               = "urban-sensors-dev"

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.
(base) PS C:\Users\Lenovo\Documents\GitHub\CD_DE\pre_entrega6\infra> terraform plan -no-color |
>> Select-String "aws_redshiftserverless|aws_vpc_endpoint.kinesis|aws_iam_role.redshift_streaming|redshift_"

# aws_iam_role.redshift_streaming will be created
+ resource "aws_iam_role" "redshift_streaming" {
# aws_iam_role_policy.redshift_streaming will be created
+ resource "aws_iam_role_policy" "redshift_streaming" {
# aws_redshiftserverless_namespace.lakehouse will be created
+ resource "aws_redshiftserverless_namespace" "lakehouse" {
# aws_redshiftserverless_workgroup.lakehouse will be created
+ resource "aws_redshiftserverless_workgroup" "lakehouse" {
# aws_security_group.redshift_serverless will be created
+ resource "aws_security_group" "redshift_serverless" {
# aws_vpc_endpoint.kinesis will be created
+ redshift_iam_role_arn      = (known after apply)
+ redshift_namespace_name   = "urban-lakehouse-dev"
+ redshift_workgroup_endpoint = (known after apply)
+ redshift_workgroup_name   = "urban-lakehouse-dev"
```

Plan Terraform completo: 22 recursos a agregar, 0 cambios y 0 destrucciones. [01_terraform_plan_redshift.txt]

Apply complete! Resources: 21 added, 0 changed, 0 destroyed.

Outputs:

```
artifact_bucket_name = "coderhouse-urban-streaming-raw-gusper-dev"
firehose_arn = "arn:aws:firehose:us-east-1:389938726295:deliverystream/ingesta-urban-sensors-dev"
firehose_name = "ingesta-urban-sensors-dev"
flink_application_name = "urban-stream-processing-dev"
glue_database_name = "lakehouse_db"
lakehouse_bucket_arn = "arn:aws:s3:::coderhouse-urban-lakehouse-gusper-dev"
lakehouse_bucket_name = "coderhouse-urban-lakehouse-gusper-dev"
redshift_iam_role_arn = "arn:aws:iam::389938726295:role/redshift-streaming-ingestion-dev"
redshift_namespace_name = "urban-lakehouse-dev"
redshift_workgroup_endpoint = tolist([
  {
    "address" = "urban-lakehouse-dev.389938726295.us-east-1.redshift-serverless.amazonaws.com"
    "port" = 5439
    "vpc_endpoint" = tolist([
      {
        "network_interface" = tolist([
          {
            "availability_zone" = "us-east-1b"
            "network_interface_id" = "eni-0064947d98b944c36"
            "private_ip_address" = "172.31.30.204"
            "subnet_id" = "subnet-4d6ae600"
          },
          {
            "availability_zone" = "us-east-1f"
            "network_interface_id" = "eni-0120d060a9213ad69"
            "private_ip_address" = "172.31.72.232"
            "subnet_id" = "subnet-258e382b"
          },
          {
            "availability_zone" = "us-east-1d"
            "network_interface_id" = "eni-01e439f52432e3d69"
            "private_ip_address" = "172.31.11.17"
            "subnet_id" = "subnet-fda35f9b"
          }
        ])
        "vpc_endpoint_id" = "vpce-03d99cff505fc3d7a"
        "vpc_id" = "vpc-1e6a6264"
      }
    ])
  }
])
redshift_workgroup_name = "urban-lakehouse-dev"
stream_arn = "arn:aws:kinesis:us-east-1:389938726295:stream/urban-sensors-dev"
stream_name = "urban-sensors-dev"
(base) PS C:\Users\Lenovo\Documents\GitHub\CD_DE\pre entrega6\infra>
```

Apply completo luego de crear previamente el bucket de artefactos: 21 recursos agregados, 0 cambios y 0 destruidos.

[02_terraform_apply_redshift.png]

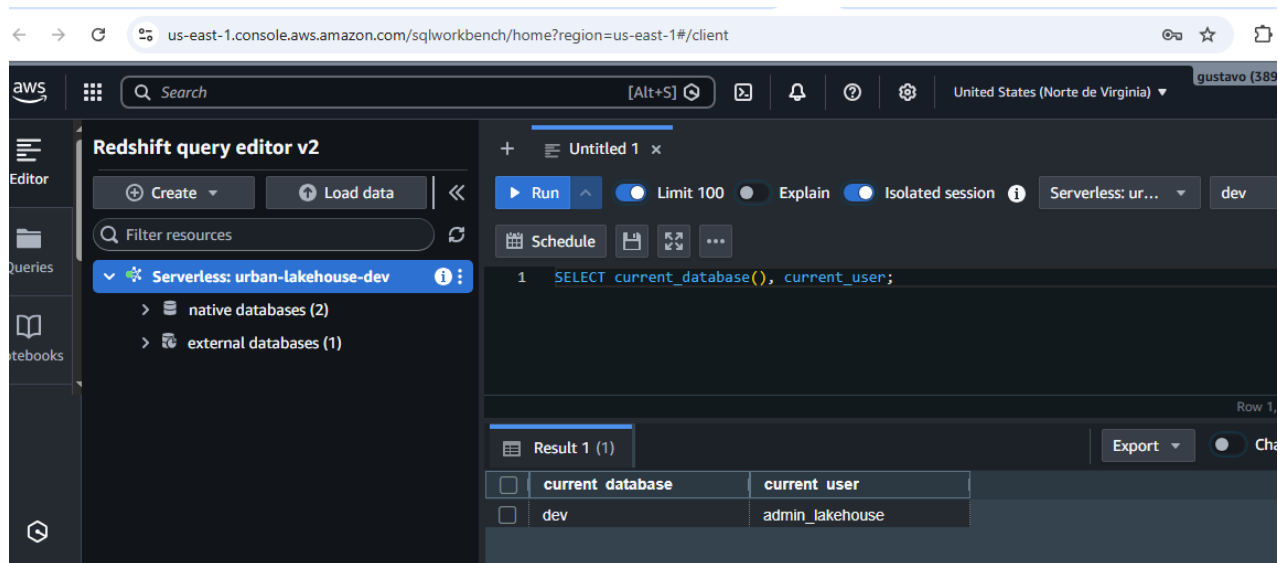
4. Disponibilidad y conexion a Redshift

El workgroup Serverless fue validado con AWS CLI antes de ejecutar SQL. Luego se utilizo Redshift Query Editor v2 conectado a la base dev con el usuario administrativo definido para el namespace.

```
(base) PS C:\Users\Lenovo\Documents\GitHub\CD_DE\pre entrega6\infra> aws redshift-serverless get-workgroup `
> --workgroup-name urban-lakehouse-dev `
> --region us-east-1 `
> --query "workgroup.{Status:status,Name:workgroupName,Endpoint:endpoint.address,BaseCapacity:baseCapacity}" `
> --output table

-----
GetWorkgroup
-----
BaseCapacity | 8
Endpoint     | urban-lakehouse-dev.389938726295.us-east-1.redshift-serverless.amazonaws.com
Name         | urban-lakehouse-dev
Status       | AVAILABLE
-----
```

Workgroup urban-lakehouse-dev en estado AVAILABLE. [03_redshift_serverless_available.png]



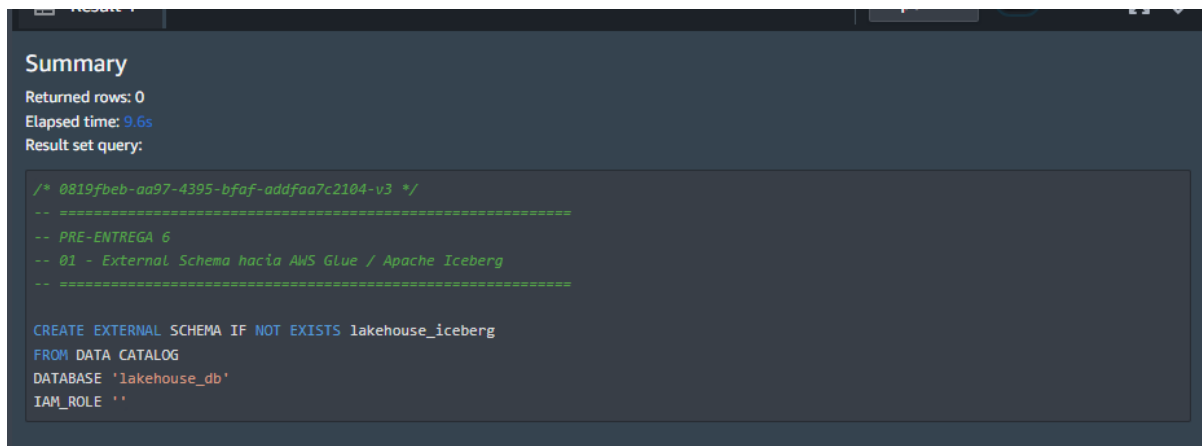
Prueba de conexion en Query Editor v2: current_database=dev y current_user=admin_lakehouse.

[04_query_editor_connection.png]

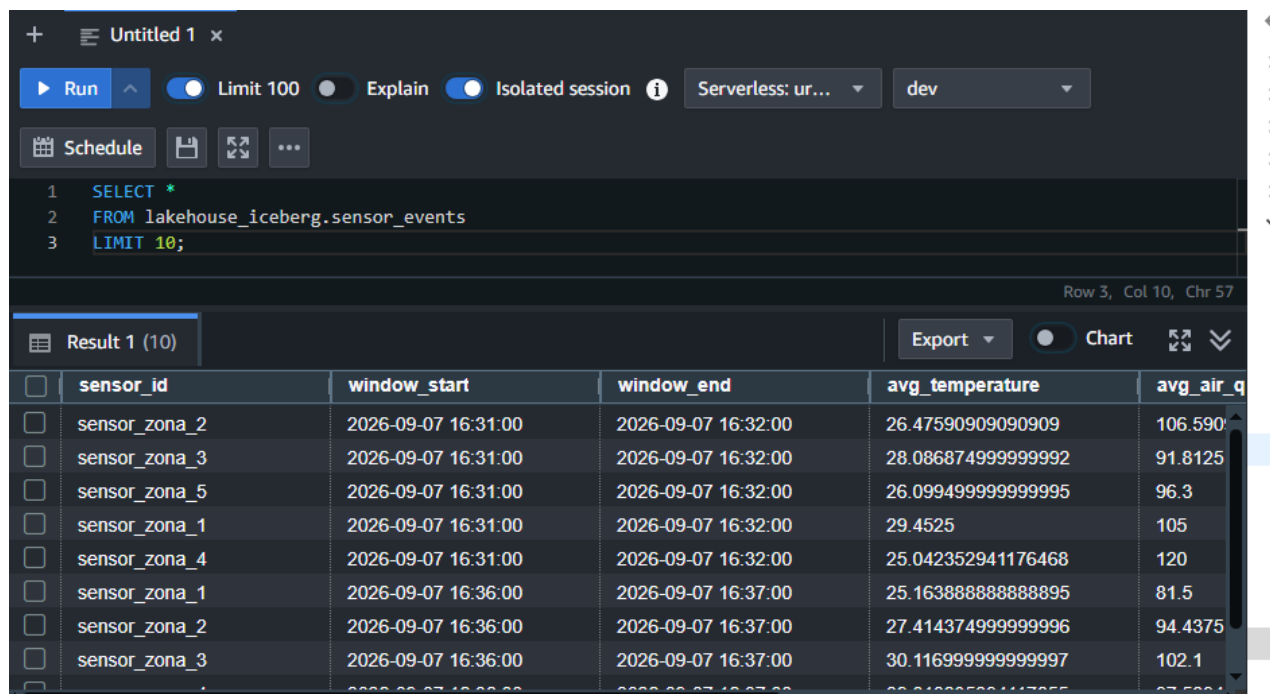
5. Integracion Lakehouse: Glue Data Catalog + Apache Iceberg

Se creo un EXTERNAL SCHEMA en Redshift que referencia la base lakehouse_db de Glue. De esta forma, Redshift puede consultar la tabla Iceberg sensor_events sin copiar los datos historicos al warehouse.

```
CREATE EXTERNAL SCHEMA IF NOT EXISTS lakehouse_iceberg
FROM DATA CATALOG
DATABASE 'lakehouse_db'
IAM_ROLE default;
```



Creacion del external schema lakehouse_iceberg en Redshift. [05_external_schema_glue.png]



Untitled 1 x

Run Limit 100 Explain Isolated session Serverless: ur... dev

Schedule

```
1 SELECT *
2 FROM lakehouse_iceberg.sensor_events
3 LIMIT 10;
```

Row 3, Col 10, Chr 57

Result 1 (10)

	sensor_id	window_start	window_end	avg_temperature	avg_air_q
☐	sensor_zona_2	2026-09-07 16:31:00	2026-09-07 16:32:00	26.47590909090909	106.590
☐	sensor_zona_3	2026-09-07 16:31:00	2026-09-07 16:32:00	28.086874999999992	91.8125
☐	sensor_zona_5	2026-09-07 16:31:00	2026-09-07 16:32:00	26.099499999999995	96.3
☐	sensor_zona_1	2026-09-07 16:31:00	2026-09-07 16:32:00	29.4525	105
☐	sensor_zona_4	2026-09-07 16:31:00	2026-09-07 16:32:00	25.042352941176468	120
☐	sensor_zona_1	2026-09-07 16:36:00	2026-09-07 16:37:00	25.163888888888895	81.5
☐	sensor_zona_2	2026-09-07 16:36:00	2026-09-07 16:37:00	27.414374999999996	94.4375
☐	sensor_zona_3	2026-09-07 16:36:00	2026-09-07 16:37:00	30.116999999999997	102.1

Consulta exitosa de lakehouse_iceberg.sensor_events con ventanas y promedios historicos. [06_iceberg_query.png]

6. Redshift Streaming Ingestion desde Kinesis

La ruta de baja latencia se implemento mediante un esquema externo de Kinesis y una Materialized View directamente asociada al stream urban-sensors-dev. La MV utiliza AUTO REFRESH YES y conserva el payload como SUPER mediante JSON_PARSE, evitando una etapa S3 intermedia para esta ruta.

```
CREATE EXTERNAL SCHEMA IF NOT EXISTS kinesis_stream
FROM KINESIS
REGION 'us-east-1'
IAM_ROLE default;

CREATE MATERIALIZED VIEW sensor_events_stream
AUTO REFRESH YES AS
SELECT approximate_arrival_timestamp, partition_key, shard_id, sequence_number,
       CASE WHEN CAN_JSON_PARSE(kinesis_data)
            THEN JSON_PARSE(kinesis_data)
            ELSE NULL END AS payload
FROM kinesis_stream."urban-sensors-dev";
```

Result 1Result 2

Summary

Returned rows: 0

Elapsed time: 10s

Result set query:

```
/* 9a38d849-6192-423b-a16d-eb957bc99da-v3 */
-- =====
-- PRE-ENTREGA 6
-- 02 - Streaming Ingestion desde Kinesis
-- =====

CREATE EXTERNAL SCHEMA IF NOT EXISTS kinesis_stream
FROM KINESIS
REGION 'us-east-1'
IAM_ROLE ''
```

Result 1Result 2

Returned rows: 0

Elapsed time: 1.4s

Result set query:

```
/* 88543d75-19fe-4ded-8328-5bc793e7dd5d-v3 */

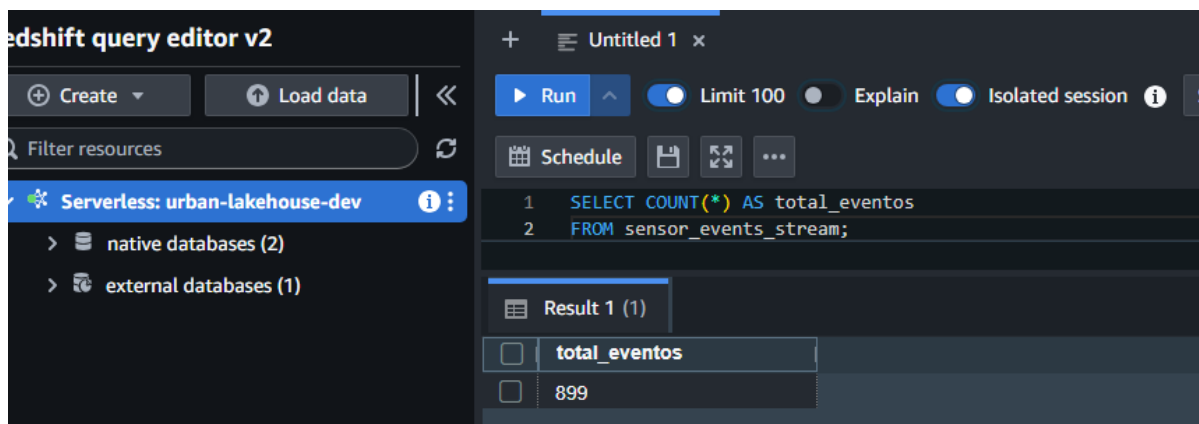
CREATE MATERIALIZED VIEW sensor_events_stream
AUTO REFRESH YES
AS
SELECT
    approximate_arrival_timestamp,
    partition_key,
    shard_id,
    sequence_number,

    CASE
        WHEN CAN_JSON_PARSE(kinesis_data)
        THEN JSON_PARSE(kinesis_data)
        ELSE NULL
    END AS payload
```

Creacion del esquema de Kinesis y de la Materialized View sensor_events_stream con AUTO REFRESH YES.
[07_streaming_ingestion_mv.png]

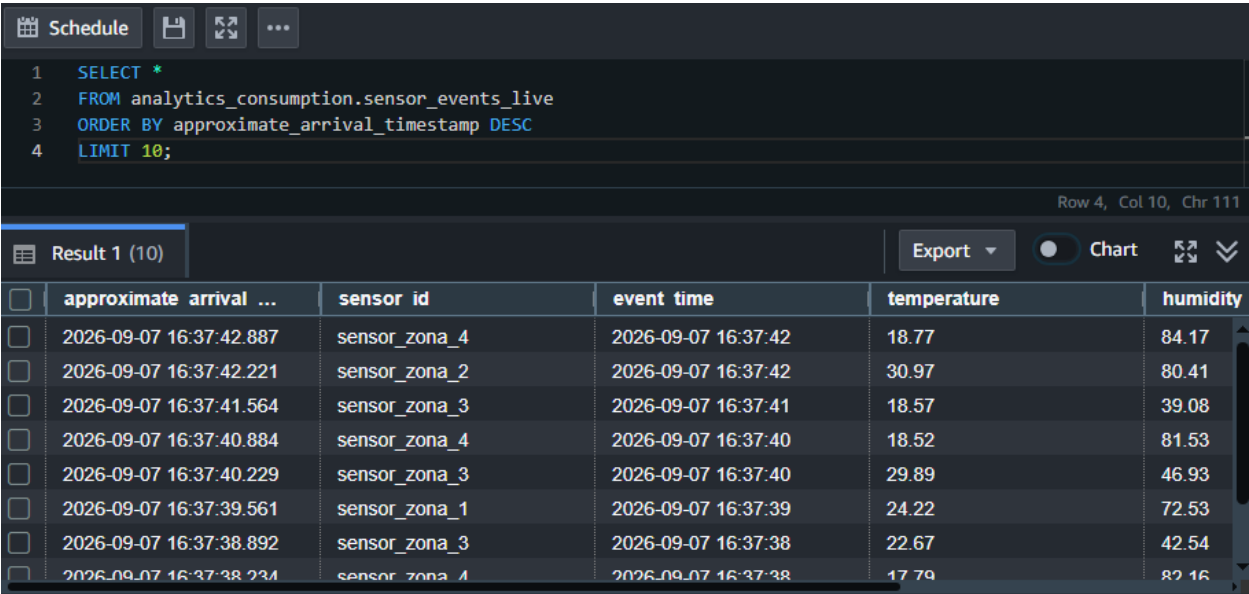
7. Datos calientes y modelado del JSON

La primera validacion funcional mostro 899 eventos en sensor_events_stream. Sobre la MV se creo una VIEW logica tipada para consumo analitico. Esta separacion es deliberada: Redshift no soporta AUTO REFRESH en una Materialized View definida sobre otra Materialized View.



Conteo inicial de 899 eventos recibidos por Streaming Ingestion. [08_kinesis_live_data.png]

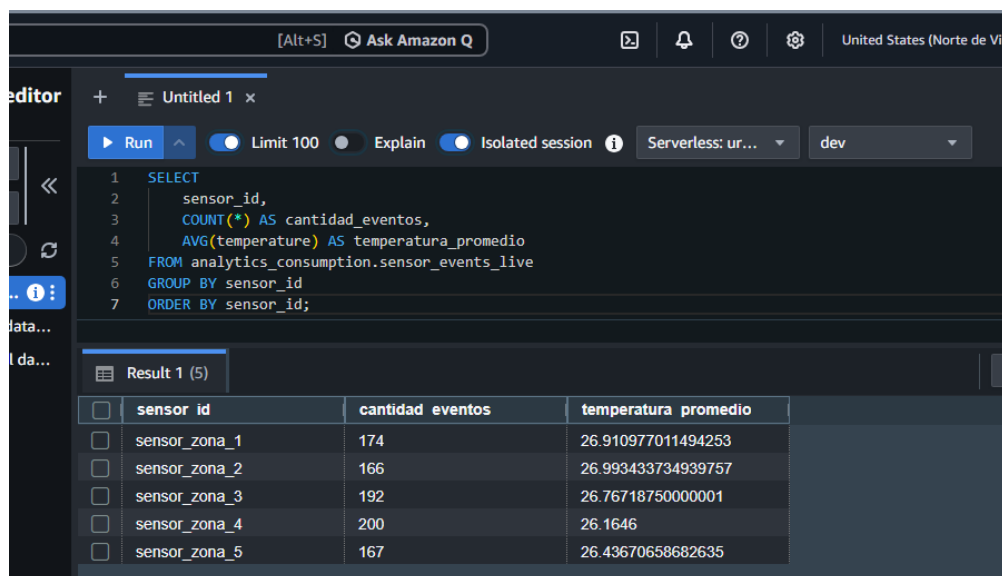
```
CREATE OR REPLACE VIEW analytics_consumption.sensor_events_live AS
SELECT approximate_arrival_timestamp,
       payload.sensor_id::VARCHAR(50) AS sensor_id,
       payload.event_time::VARCHAR(30) AS event_time,
       payload.temperature::FLOAT8 AS temperature,
       payload.humidity::FLOAT8 AS humidity,
       payload.air_quality_index::INTEGER AS air_quality_index
FROM sensor_events_stream
WHERE payload IS NOT NULL;
```



Vista tipada con sensor_id, event_time, temperature, humidity y air_quality_index. [09_parsed_live_view.png]

8. Analitica live y validacion de AUTO REFRESH

La capa de consumo permite ejecutar agregaciones de baja latencia sobre el stream. Se agruparon eventos por sensor y se calculo la temperatura promedio. Luego se dejo el producer generando eventos y se repitio el conteo sin realizar INSERT en Redshift.



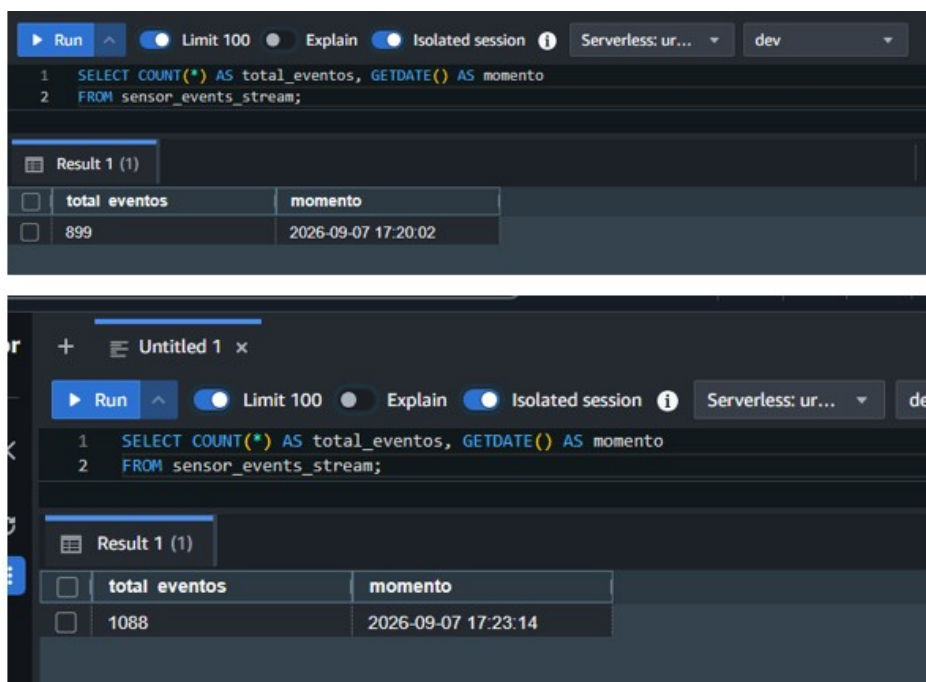
The screenshot shows the Amazon Redshift console interface. At the top, there's a search bar with "[Alt+S] Ask Amazon Q" and a location dropdown set to "United States (Norte de Vir...)". Below this is a toolbar with a "Run" button, a "Limit 100" toggle, an "Explain" toggle, an "Isolated session" toggle, and a "Serverless: ur..." dropdown. The main area displays a SQL query:

```
1 SELECT
2   sensor_id,
3   COUNT(*) AS cantidad_eventos,
4   AVG(temperature) AS temperatura_promedio
5 FROM analytics_consumption.sensor_events_live
6 GROUP BY sensor_id
7 ORDER BY sensor_id;
```

Below the query, the results are shown in a table titled "Result 1 (5)".

sensor_id	cantidad eventos	temperatura promedio
sensor_zona_1	174	26.910977011494253
sensor_zona_2	166	26.993433734939757
sensor_zona_3	192	26.767187500000001
sensor_zona_4	200	26.1646
sensor_zona_5	167	26.43670658682635

Agregacion live por sensor: cantidad de eventos y temperatura promedio. [10_live_aggregation.png]



The first screenshot shows a SQL query:

```
1 SELECT COUNT(*) AS total_eventos, GETDATE() AS momento
2 FROM sensor_events_stream;
```

The results table "Result 1 (1)" shows:

total eventos	momento
899	2026-09-07 17:20:02

The second screenshot shows the same query and results table, but the results have updated:

total eventos	momento
1088	2026-09-07 17:23:14

Prueba de auto refresh: el conteo aumento de 899 a 1088 eventos mientras el producer seguia enviando datos.

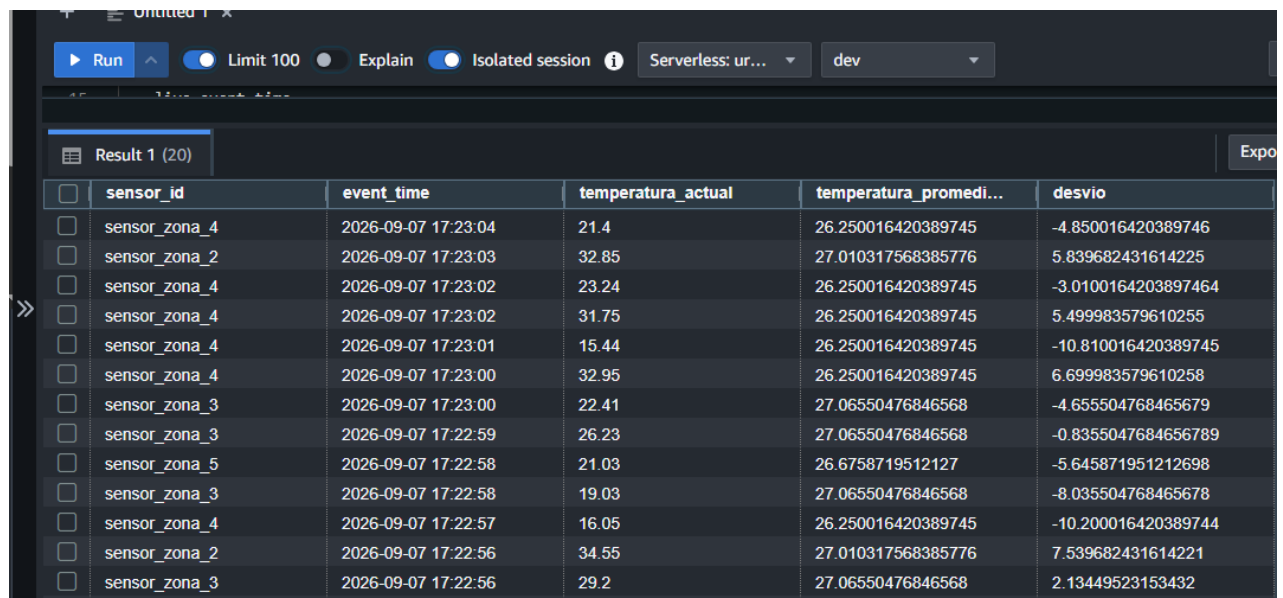
[11_mv_auto_refresh.png]

Resultado: La MV incorporo 189 eventos adicionales automaticamente. Esto valida la ingesta continua y el refresco automatico sin cargas manuales en Redshift.

9. JOIN entre datos calientes e historicos

Para cada sensor se calculo primero un promedio historico sobre la tabla Iceberg y luego se lo comparo con la temperatura live. El resultado entrega una medida de desvio entre el valor reciente y su referencia historica.

```
WITH historico AS (  
  SELECT sensor_id,  
         AVG(avg_temperature) AS temperatura_promedio_historica  
  FROM lakehouse_iceberg.sensor_events  
  GROUP BY sensor_id  
)  
SELECT live.sensor_id,  
       live.event_time,  
       live.temperature AS temperatura_actual,  
       hist.temperatura_promedio_historica,  
       live.temperature - hist.temperatura_promedio_historica AS desvio  
FROM analytics_consumption.sensor_events_live live  
JOIN historico hist ON live.sensor_id = hist.sensor_id  
ORDER BY live.approximate_arrival_timestamp DESC  
LIMIT 20;
```



	sensor_id	event_time	temperatura_actual	temperatura_promedi...	desvio
☐	sensor_zona_4	2026-09-07 17:23:04	21.4	26.250016420389745	-4.850016420389746
☐	sensor_zona_2	2026-09-07 17:23:03	32.85	27.010317568385776	5.839682431614225
☐	sensor_zona_4	2026-09-07 17:23:02	23.24	26.250016420389745	-3.0100164203897464
☐	sensor_zona_4	2026-09-07 17:23:02	31.75	26.250016420389745	5.499983579610255
☐	sensor_zona_4	2026-09-07 17:23:01	15.44	26.250016420389745	-10.810016420389745
☐	sensor_zona_4	2026-09-07 17:23:00	32.95	26.250016420389745	6.699983579610258
☐	sensor_zona_3	2026-09-07 17:23:00	22.41	27.06550476846568	-4.655504768465679
☐	sensor_zona_3	2026-09-07 17:22:59	26.23	27.06550476846568	-0.8355047684656789
☐	sensor_zona_5	2026-09-07 17:22:58	21.03	26.6758719512127	-5.645871951212698
☐	sensor_zona_3	2026-09-07 17:22:58	19.03	27.06550476846568	-8.035504768465678
☐	sensor_zona_4	2026-09-07 17:22:57	16.05	26.250016420389745	-10.200016420389744
☐	sensor_zona_2	2026-09-07 17:22:56	34.55	27.010317568385776	7.539682431614221
☐	sensor_zona_3	2026-09-07 17:22:56	29.2	27.06550476846568	2.13449523153432

Resultado del JOIN hot + historical con calculo de desvio por sensor. [\[12_hot_historical_join.png\]](#)

10. Seguridad: IAM, red privada y RBAC SQL

La seguridad se implemento en tres planos complementarios. IAM controla el acceso servicio-a-servicio; la red utiliza VPC Endpoint y Enhanced VPC Routing; y RBAC limita el acceso de usuarios al esquema de consumo.

Plano	Control aplicado
IAM	Permisos Kinesis especificos sobre el stream; lectura Glue y S3 Lakehouse. Trust para redshift.amazonaws.com y redshift-serverless.amazonaws.com.
Red	Kinesis Interface VPC Endpoint, security groups y enhanced_vpc_routing=true.
TLS	Parametro require_ssl=true en el workgroup Serverless.
RBAC SQL	REVOKE ALL sobre kinesis_stream para PUBLIC; rol analytics_reader con USAGE y SELECT solo sobre analytics_consumption.

```
REVOKE ALL ON SCHEMA kinesis_stream FROM PUBLIC;
CREATE ROLE analytics_reader;
GRANT USAGE ON SCHEMA analytics_consumption TO ROLE analytics_reader;
GRANT SELECT ON TABLE analytics_consumption.sensor_events_live TO ROLE analytics_reader;
```

```
Result 1 Result 2 Result 3 Result 4

Summary
Returned rows: 0
Elapsed time: 170ms
Result set query:

/* addba795-ad5e-48a8-b9ee-7888-73972715e1 */

--
-- ROL: analytics_reader
-- SuperUser: y: RBAC
--
-- Restrictir acceso público al esquema de Ingesta cruda
REVOKE ALL ON SCHEMA kafka_stream FROM PUBLIC;
```

```
Result 1 Result 2 Result 3 Result 4

Summary
Returned rows: 0
Elapsed time: 740ms
Result set query:

/* 74583801-a701-4734-ad6e-3b68ba7cc971 */

-- fin de consumo analítico
CREATE ROLE analytics_reader;
```

```
Result 1 Result 2 Result 3 Result 4

Summary
Returned rows: 0
Elapsed time: 1010ms
Result set query:

/* 8e1ed58e-93ee-4a28-a138-7e798-e8e027e7 */

-- Permisión acceso únicamente a la capa preparada para consumo
GRANT USAGE
ON SCHEMA analytics_consumption
TO ROLE analytics_reader;
```

```
Result 1 Result 2 Result 3 Result 4

Summary
Returned rows: 0
Elapsed time: 706ms
Result set query:

/* e877e73b-62e6-4a67-ad6f-03e75a12f8ee */

START SELECT
ON TABLE analytics_consumption.sensor_evento_live
TO ROLE analytics_reader;
```

Ejecucion de REVOKE, CREATE ROLE y GRANT para separar ingesta cruda de consumo analítico. [13_rbac_security.png]

11. Monitoreo y control operativo

Se consultaron las vistas SYS de Redshift para verificar avance del stream, errores de ingesta y refrescos de la MV. SYS_STREAM_SCAN_STATES mostro posiciones por shard; SYS_STREAM_SCAN_ERRORS devolvio 0 filas; y SYS_MV_REFRESH_HISTORY registro multiples refrescos automaticos de sensor_events_stream.

```
SELECT * FROM SYS_STREAM_SCAN_STATES
ORDER BY record_time DESC LIMIT 20;
```

```
SELECT * FROM SYS_STREAM_SCAN_ERRORS
ORDER BY record_time DESC LIMIT 20;
```

```
SELECT * FROM SYS_MV_REFRESH_HISTORY
ORDER BY start_time DESC LIMIT 20;
```

6 -- Estado de lectura del stream
7 SELECT *
8 FROM SYS_STREAM_SCAN_STATES
9 ORDER BY record_time DESC
10 LIMIT 20;

Row 10, Col 10, Chr 547

Result 1 (20)

	external_schema...	stream_name	mv_name	transaction...	query_id	record_time	partition_id	latest_positio
☐	kinesis_stream	urban-sensors-dev	sensor_events_str...	10707	613132	2026-09-07 17:23:09.7...	shardId-000000000000...	49678120084
☐	kinesis_stream	urban-sensors-dev	sensor_events_str...	10707	613132	2026-09-07 17:23:09.4...	shardId-000000000000...	49678120084
☐	kinesis_stream	urban-sensors-dev	sensor_events_str...	10702	613128	2026-09-07 17:23:07.4...	shardId-000000000000...	49678120084
☐	kinesis_stream	urban-sensors-dev	sensor_events_str...	10702	613128	2026-09-07 17:23:07.4...	shardId-000000000000...	49678120084
☐	kinesis_stream	urban-sensors-dev	sensor_events_str...	10693	613124	2026-09-07 17:23:03.5...	shardId-000000000000...	49678120084
☐	kinesis_stream	urban-sensors-dev	sensor_events_str...	10693	613124	2026-09-07 17:23:03.5...	shardId-000000000000...	49678120084
☐	kinesis_stream	urban-sensors-dev	sensor_events_str...	10686	613118	2026-09-07 17:23:01.5...	shardId-000000000000...	49678120084
☐	kinesis_stream	urban-sensors-dev	sensor_events_str...	10686	613118	2026-09-07 17:23:01.4...	shardId-000000000000...	49678120084

11
12
13 -- Errores de lectura / parseo del stream
14 SELECT *
15 FROM SYS_STREAM_SCAN_ERRORS
16 ORDER BY record_time DESC
17 LIMIT 20;

Row 14, Col 1, Chr 551

Result 1

Summary

Returned rows: 0
Elapsed time: 1.7s
Result set query:

```
/* f11b8d6f-c3ea-4bfe-bc72-f0198136e3ed-v3 */  
SELECT *  
FROM SYS_STREAM_SCAN_ERRORS  
ORDER BY record_time DESC  
LIMIT 20;
```

Run Limit 100 Explain Isolated session Serverless: ur... dev Schedule

18
19
20 -- Historial de refresh de Materialized Views
21 SELECT *
22 FROM SYS_MV_REFRESH_HISTORY
23 ORDER BY start_time DESC
24 LIMIT 20;

Row 24, Col 10, Chr 5

Result 1 (20)

	user_id	session_id	transaction_id	database_name	schema_name	mv_id	mv_name	refresh_t...
☐	100	1073783054	10707	dev	public	127492	sensor_events_stream...	Auto
☐	100	1073791247	10702	dev	public	127492	sensor_events_stream...	Auto
☐	100	1073766675	10693	dev	public	127492	sensor_events_stream...	Auto
☐	100	1073750349	10686	dev	public	127492	sensor_events_stream...	Auto
☐	100	1073946898	10675	dev	public	127492	sensor_events_stream...	Auto
☐	100	1073815835	10668	dev	public	127492	sensor_events_stream...	Auto
☐	100	1073906003	10665	dev	public	127492	sensor_events_stream...	Auto
☐	100	1073758486	10661	dev	public	127492	sensor_events stream...	Auto

Monitoreo integrado: estados de lectura, 0 errores y refresh_type=Auto para sensor_events_stream.

[14_streaming_monitoring.png]

12. Cumplimiento de criterios de evaluacion

Criterio	Evidencia de cumplimiento
Funcionalidad	Kinesis -> Redshift operativo; 899 eventos iniciales; campos JSON tipados sin errores de casting observados.
Integracion	Consulta Iceberg mediante Glue y JOIN en la misma sesion SQL entre sensor_events_live y lakehouse_iceberg.sensor_events.
Seguridad	IAM para Redshift, permisos Kinesis/Glue/S3, VPC Endpoint, require_ssl, REVOKE/GRANT y analytics_reader.
Refresco	AUTO REFRESH YES en sensor_events_stream; conteo 899 -> 1088 y SYS_MV_REFRESH_HISTORY con refresh_type Auto.
Monitoreo	SYS_STREAM_SCAN_STATES con posiciones de shards y SYS_STREAM_SCAN_ERRORS con 0 filas.
Claridad	SQL modularizado en seis scripts y consolidado en preentrega6_redshift.sql; nombres descriptivos y comentarios por bloque.

13. Decision tecnica relevante

Limitacion detectada: Redshift no permite AUTO REFRESH en una Materialized View definida sobre otra Materialized View.

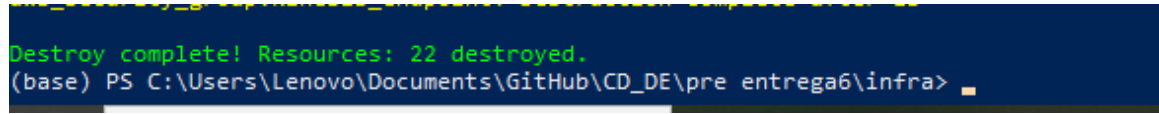
El intento inicial de crear una segunda MV tipada con AUTO REFRESH fallo por esta restriccion. La solucion final mantiene una unica MV de ingesta directamente sobre Kinesis y coloca encima una VIEW SQL normal para exponer columnas nativas. Esto conserva el refresco continuo, reduce consumidores redundantes del stream y separa correctamente ingesta de consumo.

14. Conclusiones

La Pre-entrega 6 integra satisfactoriamente la capa de consulta analitica de baja latencia con el pipeline de transporte y el Lakehouse existente. Redshift Serverless actua como consumidor directo de Kinesis para datos calientes y, simultaneamente, como motor SQL para consultar el historico Iceberg catalogado en Glue. El resultado es una arquitectura hibrida que combina latencia de segundos, persistencia historica, seguridad por capas y observabilidad operacional.

15. Cierre de infraestructura

Finalizadas las validaciones funcionales, se ejecuto terraform plan -destroy, que informo 0 recursos a crear, 0 a modificar y 22 a destruir. Luego se ejecuto terraform destroy y Terraform elimino correctamente los 22 recursos gestionados por la Pre-entrega 6.



```
Destroy complete! Resources: 22 destroyed.  
(base) PS C:\Users\Lenovo\Documents\GitHub\CD_DE\pre entrega6\infra> █
```

Destruccion completa de la infraestructura: 22 recursos eliminados. [15_terraform_destroy.png]

Verificacion posterior: terraform state list no devolvio recursos. Ademas, AWS CLI devolvio ResourceNotFoundException al consultar el workgroup urban-lakehouse-dev, la aplicacion urban-stream-processing-dev y el stream urban-sensors-dev. Esto confirma el cierre del ambiente tanto desde Terraform como desde AWS.

Anexo A. SQL consolidado utilizado

El repositorio contiene los seis scripts SQL individuales y un archivo consolidado: pre entrega6/sql/preentrega6_redshift.sql. A continuacion se reproduce la version final de los bloques funcionales.

01 - External Schema Glue/Iceberg

```
CREATE EXTERNAL SCHEMA IF NOT EXISTS lakehouse_iceberg
FROM DATA CATALOG
DATABASE 'lakehouse_db'
IAM_ROLE default;
```

02 - Streaming Ingestion Kinesis

```
CREATE EXTERNAL SCHEMA IF NOT EXISTS kinesis_stream
FROM KINESIS
REGION 'us-east-1'
IAM_ROLE default;

CREATE MATERIALIZED VIEW sensor_events_stream
AUTO REFRESH YES AS
SELECT approximate_arrival_timestamp,
       partition_key,
       shard_id,
       sequence_number,
       CASE WHEN CAN_JSON_PARSE(kinesis_data)
            THEN JSON_PARSE(kinesis_data)
            ELSE NULL END AS payload
FROM kinesis_stream."urban-sensors-dev";
```

03 - Capa analitica tipada

```
CREATE SCHEMA IF NOT EXISTS analytics_consumption;

-- Se utiliza una VIEW normal porque Redshift no admite AUTO REFRESH
-- en una Materialized View definida sobre otra Materialized View.
CREATE OR REPLACE VIEW analytics_consumption.sensor_events_live AS
SELECT approximate_arrival_timestamp,
       payload.sensor_id::VARCHAR(50) AS sensor_id,
       payload.event_time::VARCHAR(30) AS event_time,
       payload.temperature::FLOAT8 AS temperature,
       payload.humidity::FLOAT8 AS humidity,
       payload.air_quality_index::INTEGER AS air_quality_index
FROM sensor_events_stream
WHERE payload IS NOT NULL;
```

04 - JOIN hot + historical

```
WITH historico AS (
  SELECT sensor_id,
         AVG(avg_temperature) AS temperatura_promedio_historica
  FROM lakehouse_iceberg.sensor_events
  GROUP BY sensor_id
)
SELECT live.sensor_id,
       live.event_time,
       live.temperature AS temperatura_actual,
       hist.temperatura_promedio_historica,
       live.temperature - hist.temperatura_promedio_historica AS desvio
FROM analytics_consumption.sensor_events_live live
JOIN historico hist ON live.sensor_id = hist.sensor_id
ORDER BY live.approximate_arrival_timestamp DESC
```

```
LIMIT 20;
```

05 - Seguridad y RBAC

```
REVOKE ALL ON SCHEMA kinesis_stream FROM PUBLIC;  
CREATE ROLE analytics_reader;  
GRANT USAGE ON SCHEMA analytics_consumption TO ROLE analytics_reader;  
GRANT SELECT ON TABLE analytics_consumption.sensor_events_live TO ROLE analytics_reader;
```

06 - Monitoreo

```
SELECT * FROM SYS_STREAM_SCAN_STATES  
ORDER BY record_time DESC LIMIT 20;
```

```
SELECT * FROM SYS_STREAM_SCAN_ERRORS  
ORDER BY record_time DESC LIMIT 20;
```

```
SELECT * FROM SYS_MV_REFRESH_HISTORY  
ORDER BY start_time DESC LIMIT 20;
```

Anexo B. Inventario de evidencias

Nro.	Archivo	Contenido
01	01_terraform_plan_redshift.txt	Plan: 22 add, 0 change, 0 destroy.
02	02_terraform_apply_redshift.png	Apply completo: 21 added luego del bucket target.
03	03_redshift_serverless_available.png	Workgroup AVAILABLE, base capacity 8.
04	04_query_editor_connection.png	Conexion a dev como admin_lakehouse.
05	05_external_schema_glue.png	External schema Glue/Iceberg creado.
06	06_iceberg_query.png	Consulta historica Iceberg exitosa.
07	07_streaming_ingestion_mv.png	External schema Kinesis + MV AUTO REFRESH.
08	08_kinesis_live_data.png	899 eventos en la MV de streaming.
09	09_parsed_live_view.png	JSON tipado en columnas nativas.
10	10_live_aggregation.png	Agregacion live por sensor.
11	11_mv_auto_refresh.png	Conteo 899 -> 1088 sin INSERT manual.
12	12_hot_historical_join.png	JOIN live + historico con desvio.
13	13_rbac_security.png	REVOKE/ROLE/GRANT.
14	14_streaming_monitoring.png	Estados, 0 errores y refresh automatico.
15	15_terraform_destroy.png	Destroy completo: 22 recursos eliminados.

Anexo C. Base documental del modulo

- Preentrega6_consigna.pdf - requisitos del checkpoint, entregable, integracion Kinesis/Redshift/Iceberg, seguridad y criterios de evaluacion.
- GUIA_ALUMNOS_CLASE6 (1).pdf - secuencia operativa para Redshift Streaming Ingestion, Query Editor v2, pruebas de refresh y JOIN hot/historical.
- Data Engineering_01092026.pdf - marco conceptual de Streaming Ingestion vs COPY, seguridad IAM/RBAC, VPC/TLS y monitoreo.
- PODCAST DEL MODULO - EP. 05.pdf - recapitulacion del modulo: baja latencia, seguridad, lag, errores de formato y materialized views.